

Midterm_Yuliia_Chernysheva

March 6, 2026

1 CIS-2266 Midterm Project

1.1 Basketball Data Analyzer: Performance & Efficiency Study (1999-2019)

Name: Yuliia Chernysheva

Date: 3/6/2026

1.1.1 Project Description

This notebook provides an interactive analytical platform to study professional basketball statistics across various global leagues (NBA, Euroleague, etc.) for the seasons 1999-2019. The core objective is to move beyond basic scoring and calculate Analytical Efficiency Ratings to identify the most impactful athletes.

1.1.2 Inputs

- *Data Source:* A comprehensive CSV dataset (**players_stats_by_season_full_details.csv**) containing seasonal performance records.
 - *Basketball Players Stats per Season (49 Leagues) via Kaggle*
<https://www.kaggle.com/datasets/jacobbaruch/basketball-players-stats-per-season-49-leagues>
- *Context Selection:* User chooses a specific League (e.g., NBA, Euroleague) and Season Year (1999-2019).
- *Search Queries:* Specific player names for individual profiles or “Head-to-Head” comparisons.

1.1.3 Outputs

- *Standardized Player Cards:* Detailed reports showing Points (PPG), Rebounds (RPG), Assists (APG), and Efficiency.
- *Efficiency Leaderboards:* A ranked Top 10 list of the most effective players for the selected context.
- *Head-to-Head Battles:* A side-by-side comparison table that calculates the performance gap (Delta) between two selected players.

1.1.4 How to Use This Program

1. Place the **players_stats_by_season_full_details.csv** file in the same directory as this notebook.

2. Run the cells sequentially from top to bottom to initialize the Analytics Engine.
3. Follow the interactive prompts in the Main Menu:
 - *Step 1:* Define your data context (League and Year).
 - *Step 2:* Use the sub-menu to search for players, view the leaderboard, or compare athletes.
4. Type 'BACK' to change the league/season or 'EXIT' to close the program.

1.2 1) Imports & Constants

This section initializes the environment by importing necessary libraries and defining global constants. * **Centralized Constants:** Defines column names and file paths as variables to prevent typos and facilitate easy maintenance. * **Dynamic UI Layout:** Establishes relative width parameters for consistent, professional formatting across all analytical reports and leaderboards.

```
[1]: # Source: https://www.kaggle.com/datasets/jacobbaruch/
      ↪basketball-players-stats-per-season-49-leagues
import pandas as pd
from pathlib import Path
from IPython.display import display

# ----- CONSTANTS -----
# The source file
DATA_FILE = "players_stats_by_season_full_details.csv"
# Target stage to filter.
TARGET_STAGES = ('REGULAR_SEASON', 'INTERNATIONAL')

# --- COLUMN NAME CONSTANTS ---
# Using constants for column names
COL_STAGE = 'STAGE'
COL_LEAGUE = 'LEAGUE'
COL_PLAYER = 'PLAYER'
COL_TEAM = 'TEAM'
COL_SEASON = 'SEASON'
COL_GP = 'GP'           # Games Played
COL_PTS = 'PTS'         # Points
COL_REB = 'REB'         # Rebounds
COL_AST = 'AST'         # Assists
COL_FGA = 'FGA'         # Field Goals Attempted
COL_FGM = 'FGM'         # Field Goals Made
COL_TOV = 'TOV'         # Turnovers

# List of columns required for basketball performance analysis
REQUIRED_COLUMNS = [
    COL_LEAGUE, COL_PLAYER, COL_TEAM, COL_SEASON,
    COL_GP, COL_PTS, COL_REB, COL_AST,
    COL_FGA, COL_FGM, COL_TOV
]
```

```

# --- DYNAMIC UI LAYOUT ---
COL_L_WIDTH = 25  # Label width
COL_S_WIDTH = 18  # Player values width
COL_DIFF    = 10  # Difference column width
SEP_WIDTH   = 3   # Width of " | "

# Relative calculation of total width:
# Label + Player1 + Player2 + Diff + 3 delimiters
REPORT_WIDTH = COL_L_WIDTH + (COL_S_WIDTH * 2) + COL_DIFF + (SEP_WIDTH * 3)

```

1.3 2) Data Validation & Loading Functions

This module handles the initial ingestion of the dataset. It ensures that the program only proceeds if the data source is reliable and structured correctly. * **Error Handling:** Uses try-except blocks to manage missing files gracefully. * **Schema Validation:** Verifies that all mandatory columns for performance analysis (efficiency calculations) are present. * **Data Normalization:** Converts headers to uppercase to ensure case-insensitive processing. * **Strategic Filtering:** Filters records by STAGE to isolate meaningful seasonal data and prevent statistical noise.

```

[2]: def validate_dataset(df, required_list):
    """
    Checks if the uploaded CSV file contains all the necessary data columns.
    """
    # Create a list of columns that are missing in the current dataframe
    missing = [col for col in required_list if col not in df.columns]

    if missing:
        # Notify the user about specific missing data points
        print(f">> ERROR: Missing columns found: {' '.join(missing)}")
        return False

    # Success message if structure is correct
    print(">> SUCCESS: File structure is valid.")
    return True

def load_data(file_name, target_stage=TARGET_STAGES,
    ↪required_columns=REQUIRED_COLUMNS):
    """
    Reads the CSV, normalizes headers, and filters data to include only the
    ↪target stage.
    """
    try:
        # Loading the dataset using pandas
        raw_data = pd.read_csv(file_name)

        # Normalize column names to uppercase for consistency
        raw_data.columns = raw_data.columns.str.upper()

```

```

# Validate the columns before proceeding to filtration
if validate_dataset(raw_data, required_columns):
    # Filter by Stage to prevent duplicate entries per season
    clean_data = raw_data[raw_data[COL_STAGE].str.upper().
isin(target_stage)].copy()

    print(
        f">> SUCCESS: loaded {len(clean_data)} records for:"
        f"{', '.join(TARGET_STAGES)} records."
    )

    return clean_data[required_columns]
else:
    return None

except FileNotFoundError:
    print(f">> ERROR: The file '{file_name}' was not found.")
    return None

# Process data based on input
players_stat_df = load_data(DATA_FILE)

# Output results
if players_stat_df is not None:
    print("\nPreview of the selected data:")
    display(players_stat_df.head())

```

>> SUCCESS: File structure is valid.

>> SUCCESS: loaded 50974 records for:REGULAR_SEASON, INTERNATIONAL records.

Preview of the selected data:

	LEAGUE	PLAYER	TEAM	SEASON	GP	PTS	REB	AST	FGA	FGM	\
0	NBA	Shaquille O'Neal	LAL	1999 - 2000	79	2344	1078	299	1665	956	
1	NBA	Vince Carter	TOR	1999 - 2000	82	2107	476	322	1696	788	
2	NBA	Karl Malone	UTA	1999 - 2000	82	2095	779	304	1476	752	
3	NBA	Allen Iverson	PHI	1999 - 2000	70	1989	267	328	1733	729	
4	NBA	Gary Payton	SEA	1999 - 2000	82	1982	529	732	1666	747	

	TOV
0	223
1	178
2	231
3	230
4	224

1.4 3) User Input and Validation Layer

This section contains helper functions designed to manage interactive user input. The goal is to ensure that only clean, validated data reaches the analytical functions. * **Robust Error Handling:** Uses while True loops and try-except blocks to prevent the program from crashing on invalid inputs. * **Input Normalization:** Automatically applies .strip().upper() to user strings, making the search case-insensitive and resilient to accidental spaces. * **Smart Feedback:** Provides the user with “Available” hints (leagues/players) whenever an incorrect entry is made, improving the overall User Experience (UX). * **Navigation Commands:** Supports special keywords like EXIT and BACK to allow seamless navigation between different menus.

```
[3]: def prompt_season(prompt: str, min_value=1999, max_value=2019) -> int:
    """
    Prompts until user enters an appropriate Season within the given range.
    """
    while True:
        raw = input(prompt).strip()
        try:
            val = int(raw)
            if val < min_value:
                print(f">> Enter a value >= {min_value}.")
                continue
            elif val > max_value:
                print(f">> Enter a value <= {max_value}.")
                continue
            return val
        except ValueError:
            print(">> Invalid input. Enter a whole number (e.g., 2015).")

def prompt_league(prompt: str, valid_leagues) -> str:
    """
    Prompts until user enters an appropriate League name.
    """
    while True:
        user_input = input(prompt).strip().upper()

        if user_input == 'EXIT':
            return 'EXIT' # pass command back to the run_menu function

        if user_input in valid_leagues:
            return user_input
        else:
            print(f">> ERROR: No data found for {user_input}. Please try again")
            # Show a few examples of valid leagues
            print(f">> INFO: Showing 5 of {len(valid_leagues)} available_
↪leagues: "
                    f"{', '.join(valid_leagues[:5])}...")
```

```
def prompt_player(prompt: str, valid_players) -> str:
    """
    Prompts until user enters an appropriate Player name.
    """
    while True:
        # Helpful hint shown before every input attempt
        print(f">> INFO: Showing 5 of {len(valid_players)} available players: "
              f"{', '.join(valid_players[:5])}...")

        user_input = input(prompt).strip().upper()

        # FIRST CHECK: Is this an exit command?
        if user_input == 'BACK':
            return 'BACK'

        if user_input in valid_players:
            return user_input
        else:
            print(f">> ERROR: Player {user_input} not found. Please try again")
```

1.5 4) Presentation and Visualization Layer

This section defines the specialized “Printing Engines” of the application. These functions are responsible for transforming raw data dictionaries and DataFrames into human-readable, professional reports. * **Relative Design Architecture:** All tables are built using pre-defined layout constants. This ensures that changing a single width variable automatically resizes every report in the application, maintaining perfect symmetry. * **Contextual Truncation:** Implements intelligent string trimming for player names, ensuring that long names do not break the table’s alignment. * **Comparative Data Analysis:** The Comparison Table calculates “Deltas” (differences) in real-time and provides a final verdict on player performance based on the Efficiency metric. * **Visual Hierarchy:** Uses distinct separators (double and single lines) to guide the user’s eye towards key statistics and the final analytical summary.

```
[4]: def print_player_stats_card(data):
    """
    Renders a detailed player profile using relative layout constants.
    """
    line = "=" * REPORT_WIDTH
    sub_line = "-" * REPORT_WIDTH

    print(f"\n{line}")
    print(f"{'BASKETBALL ANALYTICS REPORT':^{REPORT_WIDTH}}")
    print(line)

    # Dynamic header alignment based on report width
    half_width = REPORT_WIDTH // 2
    print(f"PLAYER: {data['name']:<{half_width-9}} | TEAM: {data['team']}")
```

```

print(f" SEASON: {data['season']:<{half_width-9}} | GAMES: {data['gp']]")
print(sub_line)

# Column headers for the stats table
print(f"{'METRIC':<{COL_L_WIDTH}} | {'TOTAL':>{COL_S_WIDTH}} | {'AVG':<{COL_S_WIDTH}}")
↪>{COL_S_WIDTH}}")
print(sub_line)

stats = [
    ("Points (PTS)", data['pts'], data['ppg']),
    ("Rebounds (REB)", data['reb'], data['rpg']),
    ("Assists (AST)", data['ast'], data['apg'])
]

for label, total, avg in stats:
    print(f"{label:<{COL_L_WIDTH}} | {total:>{COL_S_WIDTH}} | {avg:<{COL_S_WIDTH}.2f}")
↪>{COL_S_WIDTH}.2f}")

print(sub_line)
# Efficiency summary row
print(
    f"{'EFFICIENCY RATING':<{COL_L_WIDTH}} | {'--':>{COL_S_WIDTH}} | "
    f"{data['eff_per_game']:>{COL_S_WIDTH}.2f}")
print(f"{line}\n")

def print_leaderboard(top_df, title="LEADERBOARD"):
    """
    Renders a clean, ranked leaderboard table for multiple players.
    """
    line = "=" * REPORT_WIDTH

    # Header
    print(f"\n{line}")
    print(f"{title:^{REPORT_WIDTH}}")
    print(line)

    # Table Headers (RK = Rank)
    print(f"{'RK':<3} | {'PLAYER':<{COL_L_WIDTH}} | {'EFF/G':>{COL_S_WIDTH}}")
    print("-" * REPORT_WIDTH)

    # Iterating through rows to display rankings
    for rank, (_, row) in enumerate(top_df.iterrows(), 1):
        name = row[COL_PLAYER]
        # Trim name if it exceeds column constraints to maintain layout
        ↪integrity
        display_name = (name[:COL_L_WIDTH-3] + '..') if len(name) > COL_L_WIDTH
        ↪else name

```

```

        print(f"{rank:<3} | {display_name:<{COL_L_WIDTH}} | "
              f"{row['EFF_PG']:>{COL_S_WIDTH}.2f}")

    # Footer
    print(line + "\n")

def print_comparison_table(p1, p2):
    """
    Renders a side-by-side comparison report using relative layout constants.
    """
    # Expanded width for the comparative layout
    line = "=" * REPORT_WIDTH
    sub_line = "-" * REPORT_WIDTH

    print(f"\n{line}")
    print(f"{'HEAD-TO-HEAD BATTLE':~{REPORT_WIDTH}}")
    print(line)

    # Trimming player names for the comparison header
    name1 = ((p1['name'][:COL_S_WIDTH-2] + '..') if len(p1['name']) >
COL_S_WIDTH
              else p1['name'])
    )
    name2 = ((p2['name'][:COL_S_WIDTH-2] + '..') if len(p2['name']) >
COL_S_WIDTH
              else p2['name'])
    )

    print(f"{'METRIC':<{COL_L_WIDTH}} | {name1:>{COL_S_WIDTH}} | "
          f"{name2:>{COL_S_WIDTH}} | {'DIFF':>8}")
    print(sub_line)

    # Data points selected for head-to-head analysis
    metrics = [
        ("Points (PPG)", p1['ppg'], p2['ppg']),
        ("Rebounds (RPG)", p1['rpg'], p2['rpg']),
        ("Assists (APG)", p1['apg'], p2['apg']),
        ("Efficiency (EFF)", p1['eff_per_game'], p2['eff_per_game'])
    ]

    for label, val1, val2 in metrics:
        delta = val1 - val2
        # Format the difference with a plus or minus sign
        delta_str = f"({delta:+.2f})"
        print(f"{label:<{COL_L_WIDTH}} | {val1:>{COL_S_WIDTH}.2f} | "
              f"{val2:>{COL_S_WIDTH}.2f} | {delta_str:>{COL_DIFF}}")

```

```

print(sub_line)
# Summarize the 'Winner' based on the Efficiency metric
leader = p1['name'] if p1['eff_per_game'] > p2['eff_per_game'] else_
↳p2['name']
print(f"ANALYTICAL ADVANTAGE: {leader.upper()}")
print(line + "\n")

```

1.6 5) Data Retrieval and Filtering Helpers

This section provides utility functions to manage and filter the dataset. It extracts unique categorical values for the user interface and isolates specific data slices (league/season) to ensure accurate analysis.

```

[5]: # Get a list of unique leagues once in uppercase
def get_valid_league(df) -> list:
    return sorted(df[COL_LEAGUE].str.upper().unique().tolist())

# Get a list of unique leagues once in uppercase
def get_valid_players(df) -> list:
    return sorted(df[COL_PLAYER].str.upper().unique().tolist())

def filter_data(df, target_league: str, session_year: int) -> pd.DataFrame:
    """
    Filters the dataset based on the selected league and season year.
    Constructs the season string (e.g., '2015 - 2016') dynamically.
    """
    target_season = f"{session_year} - {session_year + 1}"

    mask = (
        (df[COL_LEAGUE].str.upper() == target_league.upper()) &
        (df[COL_SEASON].str.upper() == target_season.upper())
    )

    return df[mask]

```

1.7 6) Analytics and Metric Extraction

The Analytics Engine defines the core mathematical logic. It features a universal efficiency calculator and a “data packager” that standardizes raw statistics into a structured format for the presentation layer.

```

[6]: def calculate_efficiency(data):
    """
    Universal efficiency calculator.
    Works with both a single player (Series) or a full table (DataFrame).
    """

```

```

# Formula: (PTS + REB + AST) - (FGA - FGM) - TOV
total_eff = (
    (data[COL_PTS] + data[COL_REB] + data[COL_AST]) -
    (data[COL_FGA] - data[COL_FGM]) -
    data[COL_TOV]
)
return total_eff

def get_player_metrics(row):
    """
    Transforms a raw DataFrame row into a standardized dictionary of metrics.
    Acts as the primary data package for the UI functions.
    """
    gp = int(row[COL_GP])
    # Calculate efficiency
    total_eff = calculate_efficiency(row)

    # Return the standardized dictionary
    return {
        'name': row[COL_PLAYER],
        'team': row[COL_TEAM],
        'season': row[COL_SEASON],
        'gp': gp,
        'pts': int(row[COL_PTS]),
        'reb': int(row[COL_REB]),
        'ast': int(row[COL_AST]),
        'ppg': row[COL_PTS] / gp if gp > 0 else 0,
        'rpg': row[COL_REB] / gp if gp > 0 else 0,
        'apg': row[COL_AST] / gp if gp > 0 else 0,
        'eff_per_game': total_eff / gp if gp > 0 else 0
    }

```

1.8 7) Application Controllers

This layer manages the application flow by coordinating player searches, leaderboard generation, and head-to-head comparisons. It bridges the gap between raw analytical data and the user interface.

```

[7]: def display_player(df, target_player: str):
    """
    Searches for a player and displays a formatted analytical summary.
    """
    # Filter the data for the specific player
    player_data = df[df[COL_PLAYER].str.upper() == target_player.upper()]

    if player_data.empty:
        print(f"\n>> INFO: Player '{target_player}' not found.")

```

```

        return

    stats_row = player_data.iloc[0]
    if (stats_row[COL_GP]) == 0:
        print(f"\n>> ERROR: Data error: {stats_row[COL_PLAYER]} has 0 games_
↳played.")
        return

    results = get_player_metrics(stats_row)
    # Calling the design function
    print_player_stats_card(results)

def display_top_players(df, top_n=10):
    """
    Calculates efficiency for the whole selection and triggers the leaderboard_
↳UI.
    """
    # Add efficiency metrics to a copy of the data
    processed_df = df.copy()
    processed_df['TOTAL_EFF'] = calculate_efficiency(processed_df)
    processed_df['EFF_PG'] = processed_df['TOTAL_EFF'] / processed_df[COL_GP]

    # Tie-breaker: if EFF is equal, the one with more total PTS wins
    top_list = processed_df.sort_values(by=['EFF_PG', COL_PTS],
↳ascending=False).head(top_n)

    # Call the specialized printer function
    display_title = f"TOP {top_n} LEADERS BY EFFICIENCY"
    # Calling the design function
    print_leaderboard(top_list, title=display_title)

def display_comparison(df, player1: str, player2: str):
    """
    Retrieves data for two players and triggers the side-by-side comparison UI.
    """
    # Filter the data for the specific player
    search1 = df[df[COL_PLAYER].str.upper() == player1.upper()]
    search2 = df[df[COL_PLAYER].str.upper() == player2.upper()]

    if search1.empty or search2.empty:
        missing = player1 if search1.empty else player2
        print(f"\n>> INFO: Player {missing} was not found in the current_
↳selection.")
        return

    # Packing data
    p1_results = get_player_metrics(search1.iloc[0])

```

```

p2_results = get_player_metrics(search2.iloc[0])
# Calling the design function
print_comparison_table(p1_results, p2_results)

```

1.9 8) Interface Controller (Main Loop)

```

[8]: def handle_sub_menu(filtered_df, league, season):
    """
    Handles the analysis operations once a specific league and season are
    ↪selected.
    """
    # Create a list of players ONLY for the current filtered context
    current_players = get_valid_players(filtered_df)

    while True:
        line = '-' * REPORT_WIDTH
        print('\n' + line)
        print(f"ANALYSIS MODE: {league.upper()} | {season}")
        print(line)
        print("1. Player Search")
        print("2. Top 10 Leaderboard")
        print("3. Head-to-Head Comparison")
        print("4. BACK (Change League/Season)")
        print("5. EXIT Program")
        print(line)

        choice = input(">> Enter your choice (1-5): ").strip()

        if choice == '1':
            # "Player Search" choice
            name = prompt_player(">> Enter player name: ", current_players)
            if name.upper() != 'BACK':
                display_player(filtered_df, name)

        elif choice == '2':
            # "Top 10 Leaderboard" choice
            display_top_players(filtered_df)

        elif choice == '3':
            # "Head-to-Head Comparison" choice
            # Duel Mode: Selecting two candidates
            name1 = prompt_player(">> Enter the FIRST player name (or 'BACK'):"
            ↪",
                                current_players)
            if name1.upper() != 'BACK':

```

```

        name2 = prompt_player(">> Enter the SECOND player name (or_
↳ 'BACK'): ",
                                current_players)
        if name2.upper() != 'BACK':
            # Are they different?
            if name1.upper() == name2.upper():
                print("\n>> ERROR: You cannot compare a player to_
↳ themselves.")
            else:
                # If everything is perfect, run the battle!
                display_comparison(filtered_df, name1, name2)

        elif choice == '4':
            print("\n>> INFO: Returning to league/season selection...")
            break # Exit sub-menu loop

        elif choice == '5':
            return "EXIT" # Propagate exit signal to the main loop

        else:
            print("\n>> ERROR: Invalid choice. Please enter a number between 1_
↳ and 5.")

    return "CONTINUE"

def run_menu(df: pd.DataFrame) -> None:
    """
    The entry point of the application. Manages initial data context selection.
    """
    # Get a list of unique leagues
    valid_leagues = get_valid_league(players_stat_df)

    while True:
        line = '=' * REPORT_WIDTH
        print('\n' + line)
        print(f"{'Basketball Data Analyzer: Seasons 1999-2019':
↳ ^{REPORT_WIDTH}}")
        print(line)
        print(f"{'STEP 1: SELECT YOUR DATA CONTEXT': ^{REPORT_WIDTH}}")
        print(line)

        # Ask for league and season ONCE at the start
        user_league = (
            prompt_league(">> Enter League (e.g., NBA, Euroleague) or 'exit' to_
↳ quit: ",
                            valid_leagues)
        )

```

```

        if user_league.upper() == 'EXIT':
            print("\n>> Goodbye!")
            break

        user_season = prompt_season(">> Enter Season 1999-2019 (e.g., 1999, 2019): ")

        # Filtering the dataframe based on the league name provided by the user
        filtered_data = filter_data(players_stat_df, user_league, user_season)

        # If filtering failed (no such league/year), the loop starts over
        if filtered_data.empty:
            print(f">> ERROR: No data found for '{user_league}' in {user_season}.")
            print(">> INFO: Please check the spelling and try again.")
            continue

        # Sub-Menu
        status = handle_sub_menu(filtered_data, user_league, user_season)

        if status == "EXIT":
            print("\n>> Goodbye!")
            break

run_menu(players_stat_df)

```

```

=====
                        Basketball Data Analyzer: Seasons 1999-2019
=====
                        STEP 1: SELECT YOUR DATA CONTEXT
=====

>> Enter League (e.g., NBA, Euroleague) or 'exit' to quit:  nb
>> ERROR: No data found for NB. Please try again
>> INFO: Showing 5 of 49 available leagues: ARGENTINIAN-LIGA-A, AUSTRALIAN-NBL,
AUSTRIAN-A-BUNDESLIGA, BALKAN-BIL, BELARUSIAN-BPL...
>> Enter League (e.g., NBA, Euroleague) or 'exit' to quit:  nba
>> Enter Season 1999-2019 (e.g., 1999, 2019):  201
>> Enter a value >= 1999.
>> Enter Season 1999-2019 (e.g., 1999, 2019):  2020
>> Enter a value <= 2019.
>> Enter Season 1999-2019 (e.g., 1999, 2019):  2019

```

ANALYSIS MODE: NBA | 2019

1. Player Search
 2. Top 10 Leaderboard
 3. Head-to-Head Comparison
 4. BACK (Change League/Season)
 5. EXIT Program
-

>> Enter your choice (1-5): 2

=====

TOP 10 LEADERS BY EFFICIENCY

=====

RK	PLAYER	EFF/G
1	Giannis Antetokounmpo	36.25
2	Luka Doncic	31.74
3	James Harden	31.51
4	Karl-Anthony Towns	29.74
5	LeBron James	29.64
6	Damian Lillard	28.38
7	Anthony Davis	27.35
8	Domantas Sabonis	26.90
9	Trae Young	26.68
10	Nikola Jokic	26.67

=====

ANALYSIS MODE: NBA | 2019

1. Player Search
 2. Top 10 Leaderboard
 3. Head-to-Head Comparison
 4. BACK (Change League/Season)
 5. EXIT Program
-

>> Enter your choice (1-5): 15

>> ERROR: Invalid choice. Please enter a number between 1 and 5.

ANALYSIS MODE: NBA | 2019

1. Player Search
2. Top 10 Leaderboard
3. Head-to-Head Comparison
4. BACK (Change League/Season)
5. EXIT Program

>> Enter your choice (1-5): 1

>> INFO: Showing 5 of 280 available players: AARON GORDON, AARON HOLIDAY, ABDEL NADER, AL HORFORD, AL-FAROUQ AMINU...

>> Enter player name: LeBron Jame

>> ERROR: Player LEBRON JAME not found. Please try again

>> INFO: Showing 5 of 280 available players: AARON GORDON, AARON HOLIDAY, ABDEL NADER, AL HORFORD, AL-FAROUQ AMINU...

>> Enter player name: LeBron James

BASKETBALL ANALYTICS REPORT

PLAYER: LeBron James	TEAM: LAL
SEASON: 2019 - 2020	GAMES: 67

METRIC		TOTAL		AVG
Points (PTS)		1698		25.34
Rebounds (REB)		525		7.84
Assists (AST)		684		10.21
EFFICIENCY RATING		--		29.64

ANALYSIS MODE: NBA | 2019

1. Player Search
2. Top 10 Leaderboard
3. Head-to-Head Comparison
4. BACK (Change League/Season)
5. EXIT Program

>> Enter your choice (1-5): 3

>> INFO: Showing 5 of 280 available players: AARON GORDON, AARON HOLIDAY, ABDEL NADER, AL HORFORD, AL-FAROUQ AMINU...

>> Enter the FIRST player name (or 'BACK'): back

ANALYSIS MODE: NBA | 2019

1. Player Search
 2. Top 10 Leaderboard
 3. Head-to-Head Comparison
 4. BACK (Change League/Season)
 5. EXIT Program
-

>> Enter your choice (1-5): 3

>> INFO: Showing 5 of 280 available players: AARON GORDON, AARON HOLIDAY, ABDEL NADER, AL HORFORD, AL-FAROUQ AMINU...

>> Enter the FIRST player name (or 'BACK'): LeBron James

>> INFO: Showing 5 of 280 available players: AARON GORDON, AARON HOLIDAY, ABDEL NADER, AL HORFORD, AL-FAROUQ AMINU...

>> Enter the SECOND player name (or 'BACK'): Giannis Antetokounmpo

=====

HEAD-TO-HEAD BATTLE				
METRIC		LeBron James	Giannis Antetoko..	DIFF
Points (PPG)		25.34	29.48	(-4.13)
Rebounds (RPG)		7.84	13.59	(-5.75)
Assists (APG)		10.21	5.62	(+4.59)
Efficiency (EFF)		29.64	36.25	(-6.61)

=====

ANALYTICAL ADVANTAGE: GIANNIS ANTETOKOUNMPO

=====

ANALYSIS MODE: NBA | 2019

1. Player Search
 2. Top 10 Leaderboard
 3. Head-to-Head Comparison
 4. BACK (Change League/Season)
 5. EXIT Program
-

>> Enter your choice (1-5): 4

>> INFO: Returning to league/season selection...

=====

Basketball Data Analyzer: Seasons 1999-2019

=====

STEP 1: SELECT YOUR DATA CONTEXT

=====

>> Enter League (e.g., NBA, Euroleague) or 'exit' to quit: Euroleague

>> Enter Season 1999-2019 (e.g., 1999, 2019): 2019

ANALYSIS MODE: EUROLEAGUE | 2019

1. Player Search
 2. Top 10 Leaderboard
 3. Head-to-Head Comparison
 4. BACK (Change League/Season)
 5. EXIT Program
-

>> Enter your choice (1-5): 2

=====

TOP 10 LEADERS BY EFFICIENCY

=====

RK	PLAYER	EFF/G
1	Shane Larkin	21.28
2	Nikola Mirotic	19.07
3	Mike James	17.39
4	Nick Calathes	17.04
5	Nikola Milutinov	16.83
6	Tornike Shengelia	16.18
7	Alexey Shved	16.12
8	Bojan Dubljevic	15.65
9	Luke Sikma	15.32
10	Greg Monroe	14.93

=====

ANALYSIS MODE: EUROLEAGUE | 2019

1. Player Search
2. Top 10 Leaderboard
3. Head-to-Head Comparison

- 4. BACK (Change League/Season)
- 5. EXIT Program

>> Enter your choice (1-5): 5

>> Goodbye!

[]: